

1653. Minimum Deletions to Make String Balanced

Solved 

Medium

 Topics

 Companies

 Hint

You are given a string `s` consisting only of characters `'a'` and `'b'`.

You can delete any number of characters in `s` to make `s` **balanced**. `s` is **balanced** if there is no pair of indices `(i, j)` such that `i < j` and `s[i] = 'b'` and `s[j] = 'a'`.

Return the *minimum* number of deletions needed to make `s` **balanced**.

Example 1:

Input: `s = "aababbab"`

Output: 2

Explanation: You can either:

Delete the characters at 0-indexed positions 2 and 6 ("aababbab" -> "aaabbb"), or

Delete the characters at 0-indexed positions 3 and 6 ("aababbab" -> "aabbbb").

Example 2:

Input: `s = "bbaaaaabb"`

Output: 2

Explanation: The only solution is to delete the first two characters.

Constraints:

- `1 <= s.length <= 105`
- `s[i]` is `'a'` or `'b'`.

Python:

class Solution:

def minimumDeletions(self, s: str) -> int:

res = len(s)

a = 0

b = 0

```
for c in s:
    a += (c == 'a')
```

```
for c in s:
    a -= (c == 'a')
    res = min(res, a + b)
    b += (c == 'b')
```

```
return res
```

JavaScript:

```
const minimumDeletions = s => {
    let res = s.length;
    let a = 0, b = 0;

    for (const c of s)
        a += (c === 'a');

    for (const c of s) {
        a -= (c === 'a');
        res = Math.min(res, a + b);
        b += (c === 'b');
    }

    return res;
};
```

Java:

```
class Solution {
    public int minimumDeletions(String s) {
        int n = s.length(), res = n;
        int a = 0, b = 0;

        for (int i = 0; i < n; i++)
            a += s.charAt(i) & 1;

        for (int i = 0; i < n; i++) {
            int c = s.charAt(i);
            a -= c & 1;
            res = Math.min(res, a + b);
            b += (c & 1) ^ 1;
        }

        return res;
    }
}
```